Executable Archaeology Before the Copilots: Reanimating Simon's 1961 Heuristic Compiler and MIT's 1984 KBEmacs

Jeff Shrager[1], Kent M. Pitman[2], and Mia Golec[3]

[1]https://jshrager.org
[2]https://github.com/netsettler
[3]Carnegie Mellon University, Pittsburgh, PA, USA

Abstract

Executable Archaeology is the practice of studying historical computational systems by running their original code, a processed called "reanimation". Although AI-assisted programming is usually considered a new invention, the idea of a program that helps write programs is nearly as old as artificial intelligence itself. Here we report on the reanimation of two historical AI programming assistants The first, Herbert Simon's Heuristic Compiler (1961), is an IPL-V program that treats programming as a sort of problem-solving, and applies means-end analysis to it. The second, KBEmacs (1984), is the primary demonstation of MIT's Programmer's Apprentice project. It let a programmer build up programs by combining standard "clichés" through a stylized English command language. Our reanimation of the Heuristic Compiler reproduces the compiled code printed in Simon's 1963 paper, and KBEmacs exaclty reproduces several of its December 1984 self-tests. In the interest emphasizing the obvious reflexivity, we tried to allow most of the detaild reanimation reported to be carried out by a modern AI-assisted programming tool, Anthropic's Claude Code, although it started from a signifiant base of existing software, including a modern IPL-V interpreter, modern Common Lisp, and an existing Common Lisp port of MIT's Lisp "Flavors" system, a historical Lisp object-oriented programming framework.

Keywords: Artificial Intelligence, Automatic Programming, AI-Assisted Programming, Digital Preservation, History of AI, Heuristic Compiler, IPL-V (Information Processing Language V), KBEmacs, Lisp Machine, MIT's Programmer's Apprentice, H. A. Simon, Software Archaeology, R. C. Waters, Executable Archaeology, Program Reamination

1 Introduction

Modern programmers increasingly work alongside AI-based tools built on large language models and trained on enormous corpora of source code. Altough the scal and fluency of the modern tools is truly novel, the aspiration of a program takes part in the work of programming is nearly as old as programming itself. In the 1950s what we now call compilation was referred to as "automatic programming" [1]. And within a few years of the founding of artificial intelligence, researchers were asking whether the methods of AI could be turned on the task of programming itself [2, 18].

This paper describes the "reanimation" of two historical systems that represent two quite different approaches to AI-assisted programming.[1] The first is Herbert A. Simon's *Heuristic Compiler*,

[1] "Reanimation" is a the process of running historical programs on emulation stacks, from code that is as close as possible to the original.[8].

written in IPL-V in 1961 and described in a RAND report that year [24] and in the *Journal of the ACM* in 1963 [26]. Simon asked "whether the task of writing a computer program can be regarded as a 'problem' in the sense in which that term is used in GPS," the General Problem Solver of Newell, Shaw, and Simon [11]. Given a description of what a routine should do, Simon's tool searched for differences between where the routine is and where the user wants it to be, and applies operators to reduce this difference.

The second AI-assisted programming tool is *KBEmacs*, the Knowledge-Based Editor (in Emacs), built at MIT's Artificial Intelligence Laboratory as the culminating demonstration of the Programmer's Apprentice project [29, 31, 17]. The user of KBEmacs chose which standard algorithms ("clichés") to combine, and directed KBEmacs in stylized English. KBEmacs supplied the details and wrote the code.

These two systems are separated by a quarter of a century, by programming language (IPL-V on 1960s mainframes versus Zetalisp on MIT Lisp Machines), and by research tradition, yet they stake out most of the conceptual territory that today's AI coding tools occupy: How a programming task is specified, where the knowledge needed to carry it out comes from, how work is divided between human and machine, and how the system checks and explains what it has done.

We approached both as exercises in what has been called *executable archaeology* [22, 10]: studying historical computational systems by reconstructing and running their original code. The Heuristic Compiler was transcribed from a 1961 line-printer listing in Simon's papers and run on the Common Lisp IPL-V interpreter built earlier to run the Logic Theorist. KBEmacs was transcribed from a four-volume January 1985 printout and run under a new Zetalisp compatibility layer for SBLC, a modern Common Lisp.[2]. Both reanimations were themselves carried out largely by an AI coding assistant: Anthropic's Claude [5], working in its command-line form, Claude Code. The division of labor is described in "Author Contributions and Use of AI".

2 Historical Background

2.1 The Heuristic Compiler

The phrase "automatic programming" has meant different things in different decades, usually whatever was just beyond the current state of the art. For the designers of FORTRAN it meant translating algebraic formulas into machine code [1]. Such translators are deterministic, and were neither then, nor are they now, considered to be AI. Newell, Shaw, and Simon's work at RAND and Carnegie Tech in the 1950s took a different view of what computers could do. Their Logic Theorist (1956) searched for proofs of theorems in *Principia Mathematica* [12, 10], and their General Problem Solver (1959) generalized these methods. [11]. Simon's 1963 summary of the method is worth quoting in full, because the Heuristic Compiler implements it almost literally:

> Given the present and desired objects, find a difference between them. Next, find an operator relevant to the difference; determine if the operator can be applied to the present object. If so, apply it. (If not, describe the objects to which it would apply and transform the present object into an object of that kind—a new "desired object.") Take the new object thus obtained as the present object and repeat the process. [26, p. 494]

A proof and a program are not as different as it might seem; a proof is a sequence of operators that transforms axioms into a theorem. In other words, a proof is a program that generates a

[2]Coincidentally, the IPL-V interpreter used to bring up Simon's code is also written in Common Lisp

theorem from axioms. In this sense (Simon continues), "a theorem-proving system can be regarded, at least formally, as a program-writing system. Conversely, if we can formulate a programming goal as a difference between a present and a desired object, we can presumably use the same processes ... to generate a program" [26, p. 495]. The same observation, in a deductive rather than heuristic setting, later became the foundation of program synthesis from theorem proving [7, 9].

Simon described the Heuristic Compiler in various publications and reports in the early 1960s. [24, 26, 27]. He was explicit that his goal was *not to build an automatic programming tool: "no claims will be made for the heuristic programming procedures described here as practical approaches to the construction of compilers. Their interest lies in what they teach us about the nature of the programming task" [26, p. 493].*

2.2 The Programmer's Apprentice

The Programmer's Apprentice (PA) project began at MIT in the mid-1970s, with Charles Rich and Howard Shrobe's Initial Report on a LISP Programmer's Apprentice [16]. These researchers were realistic about the potential capabilities of such an AI-based programming tool at that time: "Recognizing that it will be a long time before it is possible to fully duplicate an expert programmer's abilities, the project seeks to develop an intelligent assistant system ... which will help a programmer in various phases of the programming task" [31, p. 47].

Over the following decade the project developed three central ideas: Clichés *are standard computational methods, such as a linear search, a report generator, or successive approximation, each with* roles *that vary from use to use and a fixed surrounding structure.* Plans *are a language-independent representation of programs as hierarchical flowcharts with explicit data and control flow, which Rich formalized in a Plan Calculus [17]. And the* assistant approach *itself was a division of labor wherein the programmer makes design decisions and the apprentice handles the routine details. Waters's dissertation developed the analysis of programs into plans [28].*

KBEmacs was the realization of these ideas in a working editor. It ran on MIT Lisp Machines inside the Zmacs editor, and handled both Lisp and Ada. Its best-known demonstration builds an Ada report generator, `UNIT-REPAIR-REPORT`*, from five commands such as "Define a simple-report procedure UNIT-REPAIR-REPORT" and "Fill the enumerator with a chain-enumeration of UNITS and REPAIRS" [31, p. 48],[30].*

KBEmacs was admittedly a research prototype. Waters wrote that it knew "only a few dozen clichés" where a practical tool "would need to know many hundreds," and that, its ideas having been used "for essentially all [they are] worth," KBEmacs "has been mothballed" [31, pp. 54–55]. The project continued with work on the Plan Calculus, on automated reasoning, and on requirements and design apprentices, culminating in Rich and Waters's 1990 book [19].

3 Reanimating the Heuristic Compiler

3.1 The Listing

The ource used here for the heuristic compiler is a line-printer listing headed `HEURISTIC CODER 7/16/61`*, preserved in the Herbert A. Simon Papers at the Carnegie Mellon University Archives [25], located by Mia Golec and Emily Davis of CMU. This listing is dated two weeks after RAND report P-2349, and is very nearly the program that report describes. The listing is 94 pages and about 4,470 lines of code (cards). We call this "the deck". The listing also carries various hand markings indicating a handful of proposed corrections. The deck contained an executive and two compilers; a natural-language front end (a character scanner, dictionary, and parser) intended to let the compiler*

accept English definitions; and a small "world" of knowledge about IPL-V, encoded as data, that the compilers reason from and about. The listing ends with the data and had no start card (IPL-V type 5 card).

3.2 Everything Is a Description List

The central representational idea of the Heuristic Compiler is that a routine *is an object with attributes, stored as an IPL-V description list (a property list, in Lisp terms). Each attribute is one possible definition of the routine: its IPL name, its* functional description *(a process name plus arguments, such as "INSERT ( ) AT END OF ( )"), its* state description *(which cells it affects, and what each holds before and after), its flow diagram, and its IPL-V code. Below that level, an IPL instruction is itself a description list with attributes for each field, and a symbol such as* `J3` *is a description list with a region (the letter J) and a location (the integer 3). The program never handles IPL code as text; it builds code out of these structures and only prints it at the end. Simon noted that description lists gave IPL-V "expressive capabilities not readily available in more usual computer command languages" [26, p. 506].*

The executive is the paper's general compiler. It is a small means-end loop whose differences *are kinds of definitions and whose* operators *are whole compilers. It finds the closest definition for the routine, looks up the relevant compiler in a table of connections, applies it, and tests whether the routine has improved. If the routine has not improved, the executive gives up.*

The state description compiler *is the most GPS-like part of the heuristic compiler. It compares the input and output states of each affected cell, classifies the largest differences (addition, deletion, replacement, or copying), looks up an operator relevant to it, binds the operator's variables to the actual cells, and then recursively compiles two subproblems: getting from the original input state to the required state, and getting from the operator's output state to the desired output. Its knowledge of IPL-V consists of exactly two operators, LOAD and REPLACE, each itself a tiny routine with its own state description.*

The functional description compiler *works by analogy. Given a routine described as, say, "INSERT (1) AT END OF THE VALUE OF THE ATTRIBUTE (0) OF (2)," it searches the routines already compiled for one with the same* process*, here "INSERT (0) AT END OF (1)," whose code is the single primitive* `J65`*. It then adapts that routine to the new arguments: it moves inputs into working cells, compiles code to fetch each argument, recursing when an argument is itself a described computation (a "determiner"), and wraps the result in instructions that save and restore the working cells.*

3.3 Running It

The deck runs on the same Common Lisp IPL-V interpreter originally written to run the Logic Theorist [22, 23]. This had already implemented most of IPL-V and had been validated against the Logic Theorist's own historical output. Claude extended the interpreter with primitive operations needed by the Heuristic Compiler uses that were not required by the Logic Theorist, and corrected others that the new program exercised in new ways (Appendix **??***). It also added a start card. With these changes, the program runs Simon's published examples.*

Compiling from a functional description. *Given the functional description of INSERT AT END OF VALUE LIST, the program produces* `J13 J52 11W2 11W0 J10 11W1 J65 J32 0`*, character for character the code in the 1963 paper [26, pp. 499–500] (Figure 1). A print left in routine U113 shows the code growing one argument at a time, which reproduces the sequence of stages through which the paper walks its readers. In Simon's runs, as in ours, the functional description*

is supplied as a hand-coded list structure, not as typed English; the phrase is printed from it, but the compiler matches processes by symbol identity. The English front end in the deck has not yet been run (see "Next Steps").

```
FIND THE VALUE-OF THE ATTRIBUTE (0) OF (1)              <- an already-compiled routine

                                      J10   0

                                      J65   0           <- the code of INSERT (0) AT END OF (1)

                                     11W1               <- fetch argument (1), now in W1
                                      J65   0

                                     11W2               <- fetch the determiner's arguments
                                     11W0                  and apply its code (J10)
                                      J10
                                     11W1
                                      J65   0

J13
INSERT (1) AT END OF  THE VALUE-OF THE ATTRIBUTE (0) OF (2)

                                J13   J52               <- preserve working cells W0-W2
                                     11W2
                                     11W0
                                      J10
                                     11W1
                                      J65
                                      J32   0           <- restore them and exit
```

Figure 1: The 1961 Heuristic Compiler, running in 2026, compiling INSERT AT END OF VALUE LIST from its functional description. The intermediate stages are printed by the program itself (via a print in routine U113 that appears to be a debugging aid left in the deck). The final code is character for character the code printed in Simon's 1963 paper. Annotations after `<-` are ours.

Compiling from a state description. *Simon's running example of the state description compiler is J3, SET SIGNAL MINUS: a routine that replaces the top symbol of the signal cell H5 with the symbol J3 ("minus"). The state description says that H5 goes from* some symbol on top of the rest of its pushdown list *to* J3 on top of the same rest*. The compiler finds a replacement difference in a named cell; finds REPLACE relevant, with its variable bound to H5, giving `20H5`; sets up the prefix subproblem "get J3 into the communication cell," an addition, for which LOAD is relevant, giving `10J3`; finds no suffix subproblem; and assembles `10J3 20H5 0`, the code in the paper [26, p. 498].*

There is a catch, and it is the kind of thing only running the program reveals. As listed, the deck does not compile J3. *The listing's J3 has both a functional and a state description, and the executive always prefers the functional description, on the stated assumption that "it is easier to compile from a functional description than from a state description" [26, p. 502]. No already-compiled routine shares J3's process, the functional description compiler fails, the executive sees no progress, and it gives up without ever trying the state description. The paper's J3 result is obtained only when J3's functional description is removed before the run, after which the state description compiler produces exactly the published code. The 1963 listing in RM-3588-PR suggests what happened: its executive is card for card the same, but the cards that give J3 a functional description are numbered as later*

insertions. Most likely the paper's J3 came from a version in which J3 had only a state description.

Flow diagrams. *The paper's last section handles routines with branches and loops by decomposing them into* segments *joined by a flow diagram. The program reassembles routine J77 ("test whether there is a symbol equal to 0ACCUMULATOR on list 1ACCUMULATOR") from its segments, and decomposes it again into the paper's flow diagram.*

The information annexer. *RM-3588-PR's Appendix B, a small program that stores statements such as "X114 is the X33 of the X25 of X105" into description lists, either identifying an object already in memory or creating one, was also transcribed and runs. Its output matches the fragment of 1963 output that survives in the memorandum [27, 20].*

3.4 Implementation Notes

One print routine, U126, is called but absent from the 1961 listing; it was supplied from the 1963 listing in RM-3588-PR, Appendix A [27], and affects only printing. A start card for the demonstration routine T1 was added (the 1963 listing ends with exactly such a card). The 1963 state description compiler differs from the 1961 one in representing states by bare symbols where the 1961 version holds described ones. The information annexer of RM-3588-PR, Appendix B, is 409 cards.

4 Reanimating KBEmacs

4.1 The Listings

The surviving KBEmacs source is a January 1985 printout, four bound volumes of Zetalisp listings, preserved by Kent Pitman, a member of the KBEmacs team, whose scans are at `https://github.com/netsettler/netsettlement/tree/main/kbe` *[15]. These cover the system as of KBE version 358.0x (December 1984), although individual files carry their own dates and versions. Volume 1 opens with something unusual and extremely useful to an archaeologist: the results of KBEmacs's automatic self-test, run "Friday the fourteenth of December, 1984; 2:25:23 pm" on the Lisp Machine* Merlin. *The self-test runs six command sequences, compares the code KBEmacs generates with expected code, and reports that three sequences (HASH, SQRT, and SUMLIST) succeeded and three (BUCKET, DELETE, and MEMQ) failed with "Bad code generated."*

The 87 files were transcribed (mostly by ChatGPT [4]) into about 34,500 lines of source, now at `https://github.com/jeffshrager/kbemacs` *[21], roughly consistent with Waters's description of the system as "some 40,000 lines of Lisp code" [31, p. 55].*

4.2 Text and Plans

A KBEmacs user works in a stylized English command language phrased in terms of clichés and their roles. The cliché library is written as code with holes, as in the definition of successive approximation in Figure 2.

```
(DEFINE-CLICHE SUCCESSIVE-APPROXIMATION
   (PRIMARY-ROLES (INITIAL-APPROXIMATION)
    DESCRIBED-ROLES (INITIAL-APPROXIMATION CRITERION IMPROVEMENT-STEP)
    COMMENT "iteratively improves an approximation until {the criterion} is
             satisfied" ...)
  (LET* ((APPROXIMATION {the input initial-approximation}) )
    (LOOP DO
      (IF ({the criterion} APPROXIMATION) (RETURN APPROXIMATION) )
      (SETQ APPROXIMATION ({the improvement-step} APPROXIMATION) ))))
```

Figure 2: A cliché from KBEmacs's library (`kbelib.lisp`, 1984). The braces are KBEmacs read syntax marking *roles*, the parts that vary from use to use; the surrounding code is fixed.

********** *Clichés are not stored as text. When the library loads, each definition is run through KBEmacs's* analyzer*, which turns code into a plan: a set of facts in a small database describing segments, ports, data flow, control flow, and roles. In a plan, a* `LOOP` *with a* `SETQ` *disappears; what remains is a temporal composition of a sequence of states. The knowledge-based editor operates on plans, and after every command the* coder *turns the plan back into program text, deciding where to nest expressions and where to introduce variables, and preferring names the programmer has already used. Waters called this two-way translation the* representation shift*, and argued that it gave plans locality, additivity, and modularity that text lacks [31]. Because the user may also simply edit the text, KBEmacs must keep text and plan consistent in both directions.*

4.3 Running It

KBEmacs depends heavily on the MIT Lisp Machine environment, which no longer exits in a running form, inclucing Flavors (the Lisp Machine's object system), the Zmacs editor, Lisp Machine processes, Waters's own LetS loop notation, and Zetalisp itself which is similar to, but not exactly the same a modern Common Lisp. The reanimation targeted the headless *core (the database, analyzer, cliché library, command language, knowledge-based editor, and coder), leaving out the editor, windows, actors, and the Ada front end. [21]*

The 1985 source is loaded unchanged, into SBCL (`https://www.sbcl.org`), a modern Common Lisp, through a Zetalisp compatibility layer that Claude assembled and debugged. The largest single piece of that layer is a Common Lisp port of the MIT Flavors object oriented programmming system, created by g000001's for CMU's Spice Lisp [6].[3]

The rest of the layer supplies the parts of Zetalisp and the Lisp Machine that Common Lisp lacks or does differently. A small driver was also added to replay a self-test sequence from the 1984 record and compare the generated code with the expected code, as did the original test harness.

The SQRT sequence (Figure 3) builds a square-root routine from three clichés in seven commands, and then builds a variant with an extra parameter in seven more. After the first command that names a cliché, KBEmacs holds the cliché's structure with its roles still open as holes; each further command fills a role with code or with another cliché, whose own unfilled roles appear as new holes. After seven commands the plan has 32 segments and the program is complete.

[3] Coincidentally, Spice Lisp (which was built at CMU, where Simon and Newell taught!) became CMUCL, and SBCL is a fork of CMUCL!

```
>Reinitialize database.
>Define a program MYSQRT with a parameter NUM.
>Implement it as a successive-approximation.
>Implement the initial-approximation as [1.] .
>Implement the improvement-step as an average of [APPROXIMATION]
   and [(// NUM APPROXIMATION)] .
>Implement the criterion as an equality-within-epsilon of
   [(* APPROXIMATION APPROXIMATION)] and [NUM] .
>Implement the epsilon as [0.0001] .

(DEFUN MYSQRT (NUM)
  (LET* ((APPROXIMATION 1))
    (LOOP DO
      (IF (< (ABS (- (* APPROXIMATION APPROXIMATION) NUM)) 0.0001)
          (RETURN APPROXIMATION))
      (SETQ APPROXIMATION (// (+ APPROXIMATION (// NUM APPROXIMATION)) 2.0)))))

The program MYSQRT is a successive-approximation. The initial-approximation is 1.
The criterion is an equality-within-epsilon ...
```

Figure 3: KBEmacs, running in 2026, replaying the first half of the SQRT sequence from its December 14, 1984 self-test. The generated code is `equal` to the code recorded in 1984. (`//` is Zetalisp's division operator.) The final lines are KBEmacs's own description of the program, generated from its plan, as recorded in the 1984 self-test (abridged).

All fourteen commands run, and both generated programs are `equal` *to the code recorded in 1984. The generated code also works:* `(mysqrt 2.0)` *returns 1.4142157.*

4.4 Implementation Notes

The reanimation loads the headless *core of KBEmacs (database, analyzer, cliché library, command language, knowledge-based editor, and coder) in the order given by its own system definition, under SBCL. The Zetalisp compatibility layer has the following parts:*

- ***Flavors:*** *g000001's Common Lisp port of Spice Lisp Flavors [6], with a few bug fixes, extended with the Lisp Machine's* `:case` *method combination (with* `:otherwise`*),* `send` *to functions and streams, and flavor-aware* `typep`*.*
- ***LetS:*** *g000001's port of Waters's LetS series notation [6].*
- ***Reader:*** *a Zetalisp readtable (`/` as the escape character, `#/c` characters, KBEmacs's* `{...}` *and* `[...]` *syntax, and the* `...` *symbol).*
- ***Language:*** *Zetalisp has various special forms and functions that differ from Common Lisp.*

repository [21].

5 Then and Now: Historical Approaches to AI-Assisted Coding

Neither the Heuristic Compiler nor KBEmacs is an ancestor of today's AI coding assistants in any direct technical sense. Large language models are trained, not programmed with explicit operators or clichés, and they operate on program text rather than on description lists or plans. Nonetheless

the questions these two systems were built to explore are recognizably the questions that shape AI-assisted programming today.

How is a programming task specified? *Simon's two compilers differ precisely in how the task is described. A state description says what the machine looks like before and after, which is close to what we would now call a specification by examples or by pre- and post-conditions, and close to the role unit tests play in directing a modern assistant. A functional description is an imperative phrase, which is close to a natural-language prompt. Simon drew a conclusion that remains apt: programmers use both, but "at the higher levels ... a language of functions appears generally the more natural and powerful," and "the mappings from functional descriptions to process language are simpler and more direct than the mappings from state descriptions" [26, p. 506]. KBEmacs chose the functional side and gave it a vocabulary of named algorithms. Today's assistants accept both kinds of description, and much of the craft of using them lies in combining a functional request with a test-like check.*

Where does the knowledge come from? *Both historical systems are knowledge-starved by modern standards. The Heuristic Compiler's state description compiler knows exactly two operators; its functional description compiler can compile a routine only if a routine with the same process has already been compiled. KBEmacs knew "a few dozen" clichés; Waters estimated that a practical tool would need "many hundreds" [31, p. 55]. Both systems identified reuse of known solutions as the heart of the matter: Simon's functional compiler works by finding an analogous routine and adapting it, and the whole Programmer's Apprentice rests on the claim that programs are built from background knowledge. What neither could solve was acquiring that knowledge at scale. Large language models can be seen as one answer to precisely this bottleneck: an implicit cliché library absorbed from an enormous body of code, at the price of having no explicit, inspectable representation of what a cliché is or what its roles are.*

Who does what? *The Heuristic Compiler was meant to be autonomous; KBEmacs was deliberately an apprentice, leaving design decisions to the programmer and taking over "the details." Waters's argument for the apprentice was practical: full automation was not near, and a semiexpert assistant could be useful at once. Modern tools have traversed the same axis in a few years, from completing a line, to writing a function on request, to agents that plan, edit, and test across a code base with varying degrees of supervision. The question of where the programmer's decisions end and the machine's begin, central to the Programmer's Apprentice, is again a live design question.*

How does the system know it is making progress? *The Heuristic Compiler's executive is a loop: apply a compiler, test whether the routine has moved to a better kind of definition, and stop if it has not. Its state description compiler implements difference reduction in the strict GPS sense. Modern coding agents commonly run a loop of a similar shape: make a change, run the tests or the compiler, examine the difference between what happened and what was wanted, and try again. The difference detector is now an executed test suite rather than a hand-coded comparison of states, and the operator is a learned model rather than a table of connections, but the architecture of means-end iteration is recognizable. The resemblance goes beyond the loop. A coding agent of this kind is not a purely neural system: the language model chooses what to do next and interprets the results, but the processes of compiling, executing, comparing outputs, tracking changes, and so on, is done by conventional symbolic programs. In the work reported here these were a Common Lisp compiler, an IPL-V interpreter, and the 1984 self-test's own comparison of generated with expected code. Although the heuristics have moved from a hand-written table of connections into a learned model, the operators and the difference tests are still ordinary programs. KBEmacs, for its part, came with a self-test in which the expected output was code, which is exactly what made it possible to check its reanimation.*

Representation and explanation. *The two historical systems never treated code merely as*

text. The Heuristic Compiler builds IPL instructions out of description lists and prints them only at the end; KBEmacs reasons about plans and regenerates code after every command, taking pains to keep code and plan consistent when the user edits the code directly. That explicit representation is what allowed KBEmacs to describe a program in terms of its design ("The program MYSQRT is a successive-approximation . . . "). Today's assistants operate on text and also explain code fluently, but their explanations are generated from text, not read off an explicit representation of the design. The trade-off between the robustness of working with text, which KBEmacs found painfully expensive to analyze – Waters reports that "well over half of all the effort expended on KBEmacs went into implementing an analyzer and coder" [31, p. 55] – and the inspectability of working with plans, has not gone away.

==========

6 What Executable Archaeology Contributes

As with the Logic Theorist [10], it is fair to ask what running these programs teaches that reading them and their documentation does not. Four answers stand out.

It reveals gaps between the published demonstration and the surviving program. *The Heuristic Compiler's J3 example does not compile from the deck as listed; the published result requires a version of the data in which J3 lacks a functional description. This is not visible from the paper, nor from reading the executive, whose preference ordering looks perfectly sensible on the page. Similarly, the "functional descriptions" in Simon's demonstrations turn out to be hand-coded list structures; the English front end, which occupies a large part of the deck, is not exercised by any of the published examples.*

It recovers semantics that survive only in the artifacts. *The PDP-10 meaning of* `boole`*, the falsity of* `(listp nil)`*, and the integer nature of characters on the Lisp Machine are all documented somewhere, but nothing about KBEmacs's listings tells a reader that its correctness depends on them; the system simply does not work otherwise. On the IPL-V side, several J-functions had to be corrected in ways the Logic Theorist had never exercised, so each new program tests the interpreter as much as the interpreter tests the program.*

It checks the transcription, and the record. *Compensating parenthesis errors are invisible to every static check short of re-reading the scan; executing the code finds them. Conversely, the fact that a fresh transcription produces the Heuristic Compiler's published code character for character, or KBEmacs's 1984 output* `equal` *for* `equal`*, is strong evidence that the transcription and the interpreter are right along those paths, since errors would almost certainly have produced different code. Execution also provides an independent check on a transcription made by a language model.*

It makes historical claims checkable. *Simon's 1963 claims about what the Heuristic Compiler did, and the 1984 self-test's claims about what KBEmacs did, can now be tested and extended. Most interestingly, KBEmacs's self-test records three* failures*. A reanimation that reproduces those failures, with the same wrong code, would be a sharper test of fidelity than one that reproduces only the successes.*

7 Related Work

This work belongs to a series of reanimations of early AI programs from their original sources. It may be the first in which the reanimator is itself an AI program. Lane et al. restored Weizenbaum's ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094 [8]. Shrager [22]

and Moews and Shrager [10] reanimated two versions of the Logic Theorist, one in IPL-I and one in IPL-V; the IPL-V interpreter used for the Heuristic Compiler descends from that work.

The history of automatic programming has been surveyed by Balzer [2] and by Rich and Waters [18], the latter from the perspective of the Programmer's Apprentice. Deductive program synthesis, which formalizes the identification of proofs with programs that Simon made informally, begins with Green [7] and Manna and Waldinger [9]. Within the Programmer's Apprentice project, KBEmacs's capabilities and limits are assessed by Waters himself [31] and by Rich and Waters [17, 19]; Pitman describes its interface [14].

Reimplementing either system's ideas in a modern language would be a quite different undertaking from the present work. Both reanimations run the original code, transcribed from the surviving listings, on reconstructions of the abstract machine (IPL-V) or language environment (Zetalisp) it expected. For KBEmacs, a fuller reanimation would run the complete system, including its editor, on an emulated Lisp Machine; we have not attempted this.

8 Next Steps

For the Heuristic Compiler, the largest piece of unexplored territory is the English front end: the scanner, dictionary, and parser, and the routine that converts a parse into a functional description. Simon credited H. S. Kelly with "linguistic components for the compiler which are omitted from this report" [26, p. 506]; these routines may be them. Running them requires input sentences, which are not in the listing.

For KBEmacs, the next step is to run the other five sequences of the 1984 self-test, aiming to reproduce both the three successes and the three failures; then the Ada front end, which would allow the `UNIT-REPAIR-REPORT` *demonstration of Waters's papers to be rerun.*

The transcriptions, interpreters, compatibility layer, and Hacker's Guides to both programs are open source, at `https://github.com/jeffshrager/ExecutableArchaeology` *and* `https://github.com/jeffshrager/kbemacs` *[20, 21].*

Author Contributions and Use of AI

J. Shrager conceived and directed the project, searched the Simon and Newell papers in the Carnegie Mellon University Archives with M. Golec, wrote the IPL-V interpreter on which the Heuristic Compiler runs, checked the results against the historical record, and edited the paper. K. M. Pitman, a member of the original KBEmacs team, provided the scans of the January 1985 KBEmacs listings and guidance on the Zetalisp compatibility layer required to run it. M. Golec worked with Shrager over an extended period in the Simon and Newell archives, and selected and scanned the Heuristic Compiler listing. All human authors read the paper and provided edits.

Most of the programming work described here was carried out by AI systems, under the authors' direction. Most of the transcription of the listings from the page images was done by ChatGPT (OpenAI) [4]. Claude (Anthropic), running as Claude Code [5] with the Claude Opus 5.5 model, corrected the transcriptions by running the code and checking the scans; adapted and extended Shrager's existing IPL-V interpreter for the Heuristic Compiler; adapted g000001's existing Common Lisp ports of Spice Lisp Flavors and LetS [6], developed the Zetalisp compatibility layer for KBEmacs, and ran and debugged both programs until they reproduced their historical output. Claude also drafted this paper. The draft was then carefully checked by ChatGPT and by the authors, who verified its claims against the listings, the program runs, and the historical sources, and who take full responsibility for the final text.

Taken together (and excluding ChatGPT's transcription of the KBEmacs listings) the two reanimations cost about 1,500 calls to the Claude Opus 5.5 model, 1,600 tool invocations, and six hours of active machine time, spread over two weeks of calendar time, not counting ChatGPT's extraction of the KBEmacs listings from the scans.

Acknowledgments

We thank Emily Davis of Carnegie Mellon University, who, with Mia Golec, located the Heuristic Coder listing in the Herbert A. Simon Papers at the Carnegie Mellon University Archives. The Heuristic Compiler reanimation rests on Shrager's Common Lisp IPL-V interpreter, which uses David Moews's IPL-V Logic Theorist deck as its regression test. We thank g000001 for the Common Lisp port of Spice Lisp Flavors and LetS (https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS) [6]. RAND Memorandum RM-3588-PR was obtained from bitsavers (http://bitsavers.org/pdf/rand/ipl/). Listing excerpts from KBEmacs are copyright 1983/1984 Massachusetts Institute of Technology and are reproduced by permission under the notice in those listings.

Finally, we acknowledge Herbert Simon, H. S. Kelly, Richard Waters, Charles Rich, and the other members of the Programmer's Apprentice project, on whose work this paper rests.

References

[1] *J. W. Backus et al., "The FORTRAN automatic coding system," in Proc. Western Joint Comput. Conf., 1957, pp. 188–198.*

[2] *R. Balzer, "A 15 year perspective on automatic programming," IEEE Trans. Softw. Eng., vol. SE-11, no. 11, pp. 1257–1268, Nov. 1985.*

[3] *M. Chen et al., "Evaluating large language models trained on code," Jul. 2021, arXiv:2107.03374. [Online]. Available: https://arxiv.org/abs/2107.03374*

[4] *OpenAI, "ChatGPT," 2026. [Online]. Available: https://chatgpt.com*

[5] *Anthropic, "Claude Code," 2026. [Online]. Available: https://www.anthropic.com/claude-code*

[6] *g000001, "spice-flavors" and "LetS," GitHub. Accessed: Oct. 10, 2026. [Online]. Available: https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS*

[7] *C. Green, "Application of theorem proving to problem solving," in Proc. 1st Int. Joint Conf. Artif. Intell., 1969, pp. 219–239.*

[8] *R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, "ELIZA reanimated: Restoring the mother of all chatbots to one of the world's first time-sharing systems," IEEE Ann. Hist. Comput., vol. 47, no. 2, pp. 68–76, Apr.-Jun. 2025.*

[9] *Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," Commun. ACM, vol. 14, no. 3, pp. 151–165, Mar. 1971.*

[10] *D. Moews and J. Shrager, "Executable archaeology: Reanimating the Logic Theorist," IEEE Ann. Hist. Comput., submitted, 2026.*

[11] A. Newell, J. C. Shaw, and H. A. Simon, "Report on a general problem-solving program," in *Proc. Int. Conf. Inform. Process., Paris: UNESCO, 1960, pp. 256–264.*

[12] A. Newell, J. C. Shaw, and H. A. Simon, "Empirical explorations of the logic theory machine: A case study in heuristic," in *Proc. Western Joint Comput. Conf., 1957, pp. 218–230.*

[13] A. Newell, F. M. Tonge, E. A. Feigenbaum, B. F. Green Jr., and G. H. Mealy, *Information Processing Language-V Manual. Englewood Cliffs, NJ, USA: Prentice-Hall, Inc., 2nd ed., 1964.*

[14] K. M. Pitman, "Interfacing to the Programmer's Apprentice," MIT AI Lab., Cambridge, MA, USA, *Working Paper 244, Feb. 1983.*

[15] K. M. Pitman, "netsettlement: kbe," GitHub (scans of Knowledge-Based Editor, four-volume listing, Jan. 1985). Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/netsettler/netsettlement/tree/main/kbe

[16] C. Rich and H. E. Shrobe, "Initial report on a LISP programmer's apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-354, Dec. 1976. (Also IEEE Trans. Softw. Eng., vol. SE-4, no. 6, pp. 456–467, Nov. 1978.)

[17] C. Rich and R. C. Waters, "The Programmer's Apprentice project: A research overview," MIT AI Lab., Cambridge, MA, USA, AI Memo 1004, Nov. 1987.

[18] C. Rich and R. C. Waters, "Automatic programming: Myths and prospects," Computer, vol. 21, no. 8, pp. 40–51, Aug. 1988.

[19] C. Rich and R. C. Waters, The Programmer's Apprentice. New York, NY, USA: ACM Press, and Reading, MA, USA: Addison-Wesley, 1990.

[20] J. Shrager, "Executable Archaeology," GitHub (HeuristicCoder, IPL-V, and LogicTheorist directories). Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/ExecutableArchaeology

[21] J. Shrager, "kbemacs: MIT's KBEmacs from the January 1985 listings, running under SBCL," GitHub. Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/kbemacs

[22] J. Shrager, "Executable archaeology: Reanimating the Logic Theorist from its IPL-V source," Mar. 13, 2026, arXiv:2603.13514v1. [Online]. Available: https://arxiv.org/abs/2603.13514

[23] J. Shrager. "An IPL-V interpreter in Common Lisp that runs the original Logic Theorist," GitHub (archived; superseded by [20]). [Online]. Available: https://github.com/jeffshrager/IPL-V

[24] H. A. Simon, "Experiments with a heuristic compiler," RAND Corp., Santa Monica, CA, USA, Paper P-2349, Jun. 30, 1961.

[25] H. A. Simon, "Heuristic Coder 7/16/61," IPL-V program listing, Herbert A. Simon Papers, Box 14, Folder FF965, Carnegie Mellon Univ. Archives, Pittsburgh, PA, USA.

[26] H. A. Simon, "Experiments with a heuristic compiler," J. ACM, vol. 10, no. 4, pp. 493–506, Oct. 1963.

[27] *H. A. Simon, "The heuristic compiler," RAND Corp., Santa Monica, CA, USA, Memorandum RM-3588-PR, May 1963. [Online]. Available: http://bitsavers.org/pdf/rand/ipl/RM-3588-PR_The_Heuristic_Compiler_May63.pdf*

[28] *R. C. Waters, "Automatic analysis of the logical structure of programs," Ph.D. dissertation, MIT, Cambridge, MA, USA, 1978. (MIT AI Lab. Tech. Rep. AI-TR-492.)*

[29] *R. C. Waters, "KBEmacs: A step toward the Programmer's Apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-753, May 1985.*

[30] *R. C. Waters, "The Programmer's Apprentice: A session with KBEmacs," IEEE Trans. Softw. Eng., vol. SE-11, no. 11, pp. 1296–1320, Nov. 1985.*

[31] *R. C. Waters, "KBEmacs: Where's the AI?" AI Mag., vol. 7, no. 1, pp. 47–56, Spring 1986.*

About the Authors

Jeff Shrager *is a cognitive scientist and computer scientist who is currently Chief Scientist of Bennu Climate, Inc., a methane removal company. Jeff was a student of Herb Simon's at CMU in the 1980s, and taught in the SymSys program at Stanford for almost 25 years (as an adjunct). His primary intellectual project is CogLean, a Lean-based library of formal components that can be used to prove claims about cognitive models. In his spare time Jeff archive dives to find early AIs, and then works with modern AIs to reanimate them.*
Kent M. Pitman {*Kent M. Pitman to provide*}
Mia Golec {*Mia Golec to provide*}